数采工站 Lite版本 SDK(ROS2)文档

适用软件版本	v2.0.5 Kai Li
文档更新时间	2025-10-20

1. SDK 概述

天工数采工站 Lite ROS2.0 SDK由北京人形机器人创新中心开发，提供了丰富的接口，涵盖各部位电机控制等。可用于编写和部署机器人应用程序，旨在帮助开发人员快速灵活地构建自己的应用程序来精确控制和使用机器人，以满足在不同应用场景下的需求。

1.1 基本框架 - ROS2

1.2 系统环境依赖

为了确保最佳的开发体验和兼容性，建议在Ubuntu 22.04系统下进行开发。当前暂不支持在Mac和Windows系统下开发。

1.3 运行环境说明

1.3.1 整体参数配置路径

代码块

```
#全局配置参数路径
/home/ubuntu/data/param
```

1.3.2 配置参数及环境变量

- 配置参数位置：`/home/ubuntu/data/param/ros2_config.txt`

代码块

```
ROS_LOG_DIR=/home/ubuntu/logs
ROS2_BAG_DIR=/home/ubuntu/bags
FASTRTPS_DEFAULT_PROFILES_FILE=/home/ubuntu/data/param/dds_profile.xml
RMW_IMPLEMENTATION=rmw_fastrtps_cpp
ROS_DOMAIN_ID=0
```

注：

ROS_LOG_DIR：日志保存位置

ROS2_BAG_DIR：bag包保存位置

FASTRTPS_DEFAULT_PROFILES_FILE：dds配置文件，默认禁止使用wifi调试dds通信；

RMW_IMPLEMENTATION：使用的中间件类型

ROS_DOMAIN_ID：domain id

- 环境变量：`/home/ubuntu/data/param/ros2_setup.bash`

上面的配置参数在登录终端时自动导出为环境变量，也可执行以下命令手动导出

代码块

```
source /home/ubuntu/data/param/ros2_setup.bash
```

1.3.3 本体通讯节点sn号配置文件

- 出场时需要配置好通讯节点的sn号，此文件一旦配好请勿轻易改动。
- `/home/ubuntu/data/param/u2c_sn.yaml`

2. 全局值说明

2.1 数采关节id定义

- 消息名称：`MotorName`
- 数据定义位置：`MotorName.msg`
- 说明：数采的电机的命名ID定义如下。

代码块

```
# head 00 + id
uint16 MOTOR_HEAD_1 = 1
uint16 MOTOR_HEAD_2 = 2
uint16 MOTOR_HEAD_3 = 3

# arm-left 10 + id
uint16 MOTOR_ARM_LEFT_1 = 11
uint16 MOTOR_ARM_LEFT_2 = 12
uint16 MOTOR_ARM_LEFT_3 = 13
uint16 MOTOR_ARM_LEFT_4 = 14
uint16 MOTOR_ARM_LEFT_5 = 15
```

```
uint16 MOTOR_ARM_LEFT_6 = 16
uint16 MOTOR_ARM_LEFT_7 = 17

# arm-right 20 + id
uint16 MOTOR_ARM_RIGHT_1 = 21
uint16 MOTOR_ARM_RIGHT_2 = 22
uint16 MOTOR_ARM_RIGHT_3 = 23
uint16 MOTOR_ARM_RIGHT_4 = 24
uint16 MOTOR_ARM_RIGHT_5 = 25
uint16 MOTOR_ARM_RIGHT_6 = 26
uint16 MOTOR_ARM_RIGHT_7 = 27

```

3. 接口说明

3.1 本体启动

- 说明：需要在终端启动数采本体通讯节点
- 启动方式

代码块

```
cd /home/ubuntu/ros2ws
sudo su    #必须给sudo权限
source install/setup.bash
ros2 launch body_control body.launch.py
```

3.2 关节温度获取接口

- 说明：所有关节温度获取，以topic形式发布，每秒1次；
- 数据定义位置：`bodyctrl_msgs::msg::MotorStatusMsg1`
- 数据格式：

代码块

```
std_msgs/Header header
MotorStatus1[] status

MotorStatus1定义如下：
uint16 name # MotorName
float32 motortemperature
```

```
7   float32 mostemperature
```

1. 获取头部关节温度信息

话题名称：`/head/motor_status`

1. 获取双臂关节温度信息

话题名称：`/arm/motor_status`

3.3 头部关节

3.3.1 状态获取接口

1. 获取头部关节信息

- 说明：获取头部关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称：`/head/status`
- 数据定义位置：`bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式：

代码块

```
1   std_msgs/Header header
2   MotorStatus[] status
3
4   MotorStatus定义如下：
5   uint16 name    # MotorName
6   float32 pos    # rad
7   float32 speed  # rad
8   float32 current # A
9   float32 temperature  # Mos温度
10  uint32 error
```

error错误字段值定义如下：

代码块

```
1   error = 33072,      设备掉线
2   error = 33073,      关节位置超限
3   error = 1,          关节电机过温
4   error = 2,          过流
5   error = 3,          电压过低
6   error = 4,          关节mos过温
```

```
error = 5,          堵转
error = 6,          电压过高
error = 7,          缺相
error = 8,          编码器错误
```

3.3.2 控制接口

1. 位置模式

- 说明：头部关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/head/cmd_pos`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式：

代码块

```
std_msgs/Header header
SetMotorPosition[] cmds

# SetMotorPosition.msg
uint16 name  # MotorName
float32 pos  # rad
float32 spd  # rad/s
float32 cur  # A
```

2. 力位混合模式

- 说明：头部关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称：`/head/cmd_ctrl`
- 数据定义位置：`bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

代码块

```
std_msgs/Header header
MotorCtrl[] cmds

# MotorCtrl.msg
uint16 name
float32 kp
```

```
float32 kd
float32 pos
float32 spd
float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/head/cmd_vel`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

代码块

```
std_msgs/Header header
SetMotorSpeed[] cmds

#SetMotorSpeed.msg
uint16 name # MotorName
float32 spd # rad/s
float32 cur # A
```

4. 关节标零

- 说明：头部关节的标零接口。
- 控制方式：topic
- 话题名称：`/head/cmd_set_zero`
- 数据定义位置：`std_msgs::msg::String`
- 数据格式：

代码块

```
以字符串形式发送关节的id实现标零;
头部关节id: 1,2,3;
```

3.4 双臂关节

3.4.1 状态获取接口

1. 获取双臂关节信息

- 说明：获取双臂关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称：`/arm/status`
- 数据定义位置：`bodyctrl_msgs::msg::MotorStatusMsg.msg`
- 数据格式：

```
代码块
1   std_msgs/Header header
2   MotorStatus[] status
3
4   MotorStatus定义如下:
5   uint16 name    # MotorName
6   float32 pos    # rad
7   float32 speed  # rad
8   float32 current # A
9   float32 temperature
10  uint32 error
```

error错误字段值定义如下：

```
代码块
1   error = 33072,      设备掉线
2   error = 33073,      关节位置超限
3   error = 1,          关节电机过温
4   error = 2,          过流
5   error = 3,          电压过低
6   error = 4,          关节mos过温
7   error = 5,          堵转
8   error = 6,          电压过高
9   error = 7,          缺相
10  error = 8,          编码器错误
```

3.4.2 控制接口

1. 位置模式

- 说明：关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/arm/cmd_pos`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetMotorPosition.msg`

- 数据格式：

```
代码块
1   std_msgs/Header header
2   SetMotorPosition[] cmds
3
4   # SetMotorPosition.msg
5   uint16 name  # MotorName
6   float32 pos  # rad
7   float32 spd  # rad/s
8   float32 cur  # A
```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd系数。
- 控制方式：topic
- 话题名称：`/arm/cmd_ctrl`
- 数据定义位置：`bodyctrl_msgs::msg::CmdMotorCtrl.msg`
- 数据格式：

```
代码块
1   std_msgs/Header header
2   MotorCtrl[] cmds
3
4   # MotorCtrl.msg
5   uint16 name
6   float32 kp
7   float32 kd
8   float32 pos
9   float32 spd
10  float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/arm/cmd_vel`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetMotorSpeed.msg`
- 数据格式：

```
std_msgs/Header header
SetMotorSpeed[] cmds

#SetMotorSpeed.msg
uint16 name # MotorName
float32 spd # rad/s
float32 cur # A
```

4. 电流模式

- 说明：关节的电流控制接口，需要提供期望电流。
- 控制方式：topic
- 话题名称：`/arm/cmd_current`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetMotorCurTor.msg`
- 数据格式：

```
std_msgs/Header header
SetMotorCurTor[] cmds

#SetMotorCurTor.msg
uint16 name # MotorName
float32 cur_tor
uint8 ctrl_status    #当前只支持电流模式
```

5. 手臂关节标零

- 说明：双臂关节的标零接口。
- 控制方式：topic
- 话题名称：`/arm/cmd_set_zero`
- 数据定义位置：`std_msgs::msg::String`
- 数据格式：

```
以字符串形式发送关节的id实现标零;
双臂关节id:
    左臂：11---17,
    右臂：21---27;
```

3.5 因时夹爪

3.5.1 全局值说明

- 消息名称：`MotorName`
- 数据定义位置：
- 说明：各夹爪ID定义如下：

```
代码块
1   uint16 MOTOR_GRIPPER_LEFT = 18,          //左夹爪
2   uint16 MOTOR_GRIPPER_RIGHT = 28,            //右夹爪
```

3.5.2 状态获取接口

- 说明：获取夹爪的状态信息，其中包含当前位置、力矩。
- 控制方式：topic
- 话题名称：`/inspire_gripper/left_status` 和 `/inspire_gripper/right_status`
- 数据定义位置：`sensor_msgs::msg::JointState.msg`
- 数据格式：

⚠ 注意：这里的数据（位置，速度，电流），指的是百分比

```
代码块
1   std_msgs/Header header
2   string[] name
3   float64[] position      #位置百分比
4   float64[] effort        #力矩百分比
```

3.5.3 错误/状态码获取接口

- 说明：获取夹爪的状态信息，其中包含当前位置、力矩。
- 控制方式：topic
- 话题名称：`/inspire_gripper/left_error` 和 `/inspire_gripper/right_error`
- 数据定义位置：`std_msgs::msg::UInt32`
- 数据格式：

```
std_msgs/Header header
Uint32 data
```

data字段值定义如下：

代码块

```
data = 33072,      设备掉线
data = 0,          没有错误
data = 1,          夹爪张开到最大且停止
data = 2,          夹爪闭合到最小且停止
data = 3,          夹爪停止
data = 4,          夹爪正在夹取
data = 5,          夹爪正在张开
data = 6,          夹爪夹到物体停止
data = 11,         堵转故障
data = 12,         过温故障
data = 13,         过流故障
data = 14,         内部通讯故障
data = 15          未知错误
```

3.5.4 控制接口

- 说明：因时夹爪的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/inspire_gripper/left_command`
  和 `/inspire_[illegible]`
- 数据定义位置：`sensor_msgs::msg::JointState`
- 数据格式：

⚠ 注意：这里的数据（位置，速度，电流），指的是百分比

代码块

```
std_msgs/Header header
float64[] position      #位置百分比
float64[] velocity      #速度百分比
float64[] effort        #电流百分比
```

3.5.5 控制夹爪模式

- 说明：因时夹爪的模式设置。
- 控制方式：topic

- 话题名称：`/inspire_gripper/set_catch_mode_command`
- 数据定义位置：`bodyctrl_msgs::msg::CmdSetParam`
- 数据格式：

```
std_msgs/Header header
SetParam[] cmds
```

```
uint16 name  //夹爪ID
float32 param
```

3.5.6 清除夹爪错误

- 说明：因时夹爪的错误清除（部分可清）。
- 控制方式：topic
- 话题名称：`/inspire_gripper/clear_error_command`
- 数据定义位置：`std_msgs::msg::String`
- 数据格式：

```
以字符串形式发送，控制双爪清除错误；
左夹爪：left
右夹爪：right
双夹爪：all
```

3.6 灵巧手

3.6.1 全局值说明

- 消息名称：`MotorName`
- 数据定义位置：
- 说明：手指各关节ID定义如下：

```
uint16 MOTOR_FINGER_LITTLE = 1,          //little finger   小指
uint16 MOTOR_FINGER_RING = 2,            //ring finger   无名指
```

```
uint16 MOTOR_FINGER_MIDDLE = 3,         //middle finger   中指
uint16 MOTOR_FINGER_FORE = 4,           //Fore finger    食指
uint16 MOTOR_FINGER_THUMB_BEND = 5,     //thumb bend   拇指弯曲
uint16 MOTOR_FINGER_THUMB_ROTATION = 6, //thumb rotation   拇指旋转
```

3.6.2 状态获取接口

- 说明：获取手指各关节的状态信息，其中包含电机的当前位置、速度、电流。
- 控制方式：topic
- 话题名称：`/inspire_hand/state/left_hand` 和 `/inspire_hand/state/right_hand`
- 数据定义位置：`sensor_msgs::msg::JointState`
- 数据格式：

⚠ 注意：这里的数据（位置，速度，电流），指的是百分比

代码块

```
std_msgs/Header header
string[] name           #说明，id=1为小指，id=6为大拇指旋转
float64[] position      #百分比
float64[] velocity      #百分比
float64[] effort        #百分比
```

3.6.3 控制接口-topic

1. 控制手指各关节角度

- 说明：手部电机的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：`/inspire_hand/ctrl/left_hand` 和 `/inspire_hand/ctrl/right_hand`
- 数据定义位置：`sensor_msgs::msg::JointState.msg`
- 数据格式：

⚠ 注意：这里的position，指的仍然是手指张开的百分比；

代码块

```
std_msgs/Header header
string[] name
float64[] position      #百分比
```

3.6.4 控制接口-service

1. 控制手指各关节角度

- 说明：手指各个关节的位置控制接口，以百分比的方式。
- 控制方式：service
- 服务名称：`/inspire_hand/set_angle_flexible/left_hand` 和 `/inspire_hand/set_angle_flexible/right_hand`
- 数据定义位置：`bodyctrl_msgs::srv::set_angle_flexible.srv`
- 数据格式：

```
代码块
1   string[] name
2   float32[] angle_ratio
3   bool angle_accepted
```

3.6.5 控制手指各关节力矩

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称：`/inspire_hand/set_force/left_hand` 和 `/inspire_hand/set_force/right_hand`
- 数据定义位置：`bodyctrl_msgs::srv::set_force.srv`
- request：

```
代码块
1   float32 force0_Ratio   #小拇指-百分比（0代表0g，1代表1000g）
2   float32 force1_Ratio   #无名指-百分比
3   float32 force2_Ratio   #中指-百分比
4   float32 force3_Ratio   #食指-百分比
5   float32 force4_Ratio   #大拇指弯曲-百分比
6   float32 force5_Ratio   #大拇指旋转-百分比
```

- response：

```
代码块
1   bool force_accepted   #是否设置成功
```

3.6.6 控制手指各关节速度

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称：`/inspire_hand/set_speed/left_hand` 和 `/inspire_hand/set_speed/right_hand`
- 数据定义位置：`bodyctrl_msgs::srv::set_speed.srv`
- request：

```
代码块
1   float32 speed0_Ratio   #小拇指-百分比（1代表800ms从最大角度到最小角度，0.5代表1600ms，0.25代表3200ms）
2   float32 speed1_Ratio   #无名指-百分比
3   float32 speed2_Ratio   #中指-百分比
4   float32 speed3_Ratio   #食指-百分比
5   float32 speed4_Ratio   #大拇指弯曲-百分比
6   float32 speed5_Ratio   #大拇指旋转-百分比
```

- response：

```
代码块
1   bool speed_accepted   #是否设置成功
```

3.6.7 清除手指各关节错误

- 说明：清除手指各个关节的错误接口。
- 控制方式：service
- 服务名称：`/inspire_hand/set_clear_error/left_hand` 和 `/inspire_hand/set_clear_error/right_hand`
- 数据定义位置：`bodyctrl_msgs::srv::set_clear_error.srv`
- request：`null`
- response：

```
代码块
1   bool setclear_error_accepted   #是否设置成功
```

3.7 六维力

3.7.1 状态获取接口

1. 六维力信息

- 说明：获取六维力当前的信息。其中上报频率为1000Hz。
- 控制方式：topic
- 话题名称：`/arm_6dof_left` 和 `/arm_6dof_right`
- 类型：`geometry_msgs::msg::WrenchStamped.msg`
- 数据格式：

代码块

```
Header header uint32 seq #序列号
Header header time stamp #时间戳
Header header string frame_id #帧id

Wrench wrench Vector3 force #3个方向的力
Wrench wrench Vector3 torque #3个方向的力矩
```

3.8 头部深度相机

启动相机方法-ROS2

代码块

```
cd /home/ubuntu/orbbec_camera_ros2
source install/setup.bash
ros2 launch orbbec_camera gemini_330_series.launch.py
```

3.9 手眼相机

启动相机方法-ROS2:

启动相机1

代码块

```
cd /home/ubuntu/real_sense_ros2
source install/setup.bash
ros2 launch realsense2_camera rs_launch.py serial_no:="'207322251310'"
camera_name:='camera1' camera_namespace:='camera1'
```

启动相机2

代码块

```
cd /home/ubuntu/real_sense_ros2
source install/setup.bash
ros2 launch realsense2_camera rs_launch.py serial_no:="'234422060144'"
camera_name:='camera2' camera_namespace:='camera2'
```

注意：`serial_no`号需要使用

代码块

```
rs-enumerate-devices -s
```

查看后，修改对应serial_no

3.10 同构臂

3.10.1 同构臂通讯节点sn号配置文件

- 出场时需要配置好同构臂通讯节点的sn号，此文件一旦配好请勿轻易改动。
- `/home/ubuntu/data/param/little_arm_u2c_sn.yaml`

3.10.2 同构臂启动

- 说明：如需使用同构臂，需要在终端启动同构臂通讯节点以及控制程序
- 启动方式1
- 启动：同构臂通讯节点

代码块

```
cd /home/ubuntu/LittleArmControl
sudo su    #必须给sudo权限
source install/setup.bash
ros2 launch little_arm_control little_arm.launch.py
```

- 启动方式2
- 启动：同构臂标定节点。（给同构臂各关节的编码器标零，否则操作双臂轨迹不可预测）

代码块

```
cd /home/ubuntu/LittleArmControl
source install/setup.bash
ros2 launch little_arm_control little_arm_calibrate.launch.py
```

注意：如果启动标定节点报错，请执行下面两条指令

代码块

```
sudo apt update
sudo apt install dbus-x11
```

- 启动方式3
- 启动：同构臂控制数采双臂及夹爪节点。(启动此节点后，同构臂会控制双臂运动，与其保持一致，随后开始同步运动)

代码块

```
cd /home/ubuntu/LittleArmControl
source install/setup.bash
ros2 launch little_arm_control little_arm_control.launch.py
```

- 启动方式4
- 整体启动：包括同构臂通讯节点、双臂及夹爪控制节点。(启动此节点后，同构臂会控制双臂运动，与其保持一致，随后开始同步运动)

代码块

```
cd /home/ubuntu/LittleArmControl
sudo su    #必须给sudo权限
source install/setup.bash
ros2 launch little_arm_control little_arm_whole.launch.py
```

3.10.3 同构臂状态读取

1. 获取同构臂状态信息

- 说明：获取同构臂的状态信息，其中包含关节的名称和当前位置。
- 控制方式：topic
- 话题名称：`/encoder_identical_joint`
- 数据定义位置：`sensor_msgs::msg::JointState.msg`
- 数据格式：

代码块

```
std_msgs/Header header
string[] name
float64[] position     #位置
float64[] velocity     #速度
```